Point of Sale System

Database Setup Guide - from zero to 23 tables

Microsoft SQL Server Express 2025 + Visual Studio Code. Nothing else is needed.

Contents. Part 1: set up the server, the VS Code extension and the connection, then build and verify the database. Part 2: every table with its attributes, types and constraints. Part 3: the SQL command for each data type and how each constraint is written.

Part 1 - Setup from zero

Step 1. Install SQL Server 2025 Express

- Open **https://www.microsoft.com/en-us/sql-server/sql-server-downloads** and find the **SQL Server 2025 Express** card (bottom right of the page). Click **Download now** and run the installer.
- Choose **Basic**, accept the license and the default folder, and wait until it says **Installation has completed successfully**.
- On that screen note: instance name **SQLEXPRESS**; SQL administrator = your Windows account; version 17.0.x.
- Click the copy button next to the **connection string** and save it in a notes file (the installer will not show it again).
- Click **Close**. You do not need "Install SSMS".
- Check the server is running: press Win+R, type `services.msc`, and look for **SQL Server (SQLEXPRESS)** with status *Running*.

A Basic install lets only your own Windows account log in. That is what we want for local work: every teammate installs their own copy.

Step 2. Install the SQL Server extension in VS Code

- In VS Code open Extensions (Ctrl+Shift+X), search **SQL Server (mssql)**, and install the one published by **Microsoft**.
- A **SQL Server** icon appears in the left bar. Installing the server and the extension does not connect them; the next step does.

Step 3. Create the connection

Click the SQL Server icon, choose **Add Connection** (or Ctrl+Shift+P, *MS SQL: Add Connection*) and fill the form:

Field	Value
Profile Name	`pos_local (any name)`
Input type	`Parameters`
Server name	`localhost\SQLEXPRESS`
Port box (shows 1433)	`leave as is; a named instance ignores it`
Trust server certificate	`checked (SQL Server 2025 encrypts by default and a local install uses a self-signed certificate)`
Authentication type	`Integrated (Windows Authentication) - not "SQL Login"`
User name / Password	`disappear when Integrated is selected`
Database name	`leave the default (master) for now`
Encrypt	`leave as Mandatory`

Click **Connect**. The profile appears under *Connections*. If it fails: tick *Trust server certificate* for a certificate error; try `.\SQLEXPRESS` for "server not found"; confirm the service is running.

Step 4. Build the database

- Get the repo (`git clone <repo-url>`) and open its folder in VS Code (File > Open Folder).
- Open **db/schema.sql**. Press Ctrl+Shift+P, run *MS SQL: Connect*, and choose **pos_local**.
- Press **Ctrl+Shift+E** (or right-click > Execute Query). The **Query Results** panel at the bottom should end with *Commands completed successfully*.
- Tables are created parents first, because a foreign key can only point to a table that already exists. The order is the tier order used in Part 2.

GO is only a batch separator for the tools, not SQL. The file starts with `CREATE DATABASE pos_store` and `USE pos_store`, so it works even though the connection starts in *master*. If you ever run pieces by hand, make sure the database shown above the editor is **pos_store**, otherwise the tables land in *master*.

Step 5. Verify

- Open **db/verify.sql**, connect it to **pos_local**, and run it.
- The key results to look for: **23 tables** and the list of all foreign keys (child table to parent table). The queries below are the core checks.
- You can also browse visually: SQL Server panel > pos_local > Databases > pos_store > Tables (right-click > Refresh).

```
SELECT COUNT(*) AS tables_created FROM sys.tables;          -- expect 23
SELECT name FROM sys.tables ORDER BY name;                  -- the 23 names
SELECT OBJECT_NAME(parent_object_id) AS child_table,
       OBJECT_NAME(referenced_object_id) AS parent_table
FROM sys.foreign_keys ORDER BY child_table;                 -- every relationship
```

Rebuilding. To start over, drop the tables children first (reverse of the tier order in Part 2) and run schema.sql again. Dropping a parent before its child fails with "referenced by a FOREIGN KEY constraint".

Part 2 - Tables: attributes, types and constraints

23 tables, listed in creation order. Every table is shown as **Attribute / Type / Constraint**. All single-column IDs are `INT IDENTITY(1000000,1)`: SQL Server numbers the rows itself, always with 7 digits (1000000 to 9999999), and a CHECK rejects anything else. Primary keys guarantee no duplicate IDs inside a table; each table has its own counter. No foreign key cascades deletes (default NO ACTION).

Renamed because of T-SQL: TRANSACTION is **Sales_Transaction**; Condition is **Product_condition**; Street# and Apt# are **Street_number** and **Apt_number**; Date (of a transaction) is **Transaction_date**; Type (of a discount) is **Discount_type**.

Tier 1 - no dependencies on other tables

1. Employee

Store staff. Each employee is a Manager or a Cashier; managers supervise other employees.

Attribute	Type	Constraint
Employee_id	`INT IDENTITY(1000000,1)`	PRIMARY KEY NOT NULL Auto-number starting at 1000000, +1 each row CHECK: 7 digits (1000000 to 9999999)
First_name	`NVARCHAR(50)`	NOT NULL
Last_name	`NVARCHAR(50)`	NOT NULL
Email	`VARCHAR(100)`	NOT NULL UNIQUE
Phone_number	`VARCHAR(20)`	NULL allowed (optional)
Role	`VARCHAR(20)`	NOT NULL CHECK: Role IN ('Manager','Cashier')
Hire_date	`DATE`	NOT NULL
Password_hash	`VARCHAR(255)`	NOT NULL
Supervisor_id	`INT`	NULL allowed (optional) CHECK: Supervisor_id <> Employee_id FOREIGN KEY -> Employee(Employee_id)

2. Rewards_Program

Defines how customers earn and redeem loyalty points.

Attribute	Type	Constraint
Program_id	`INT IDENTITY(1000000,1)`	PRIMARY KEY NOT NULL Auto-number starting at 1000000, +1 each row CHECK: 7 digits (1000000 to 9999999)
Name	`NVARCHAR(100)`	NOT NULL
Points_per_dollar	`DECIMAL(6,2)`	NOT NULL CHECK: Points_per_dollar >= 0
Redemption_rate	`DECIMAL(8,4)`	NOT NULL CHECK: Redemption_rate >= 0
Effective_from	`DATE`	NOT NULL
Effective_to	`DATE`	NULL allowed (optional) CHECK: Effective_to IS NULL OR Effective_to >= Effective_from

3. Sale_Event

A named group of discounts running over a date range.

Attribute	Type	Constraint
Sale_event_id	INT IDENTITY(1000000,1)	PRIMARY KEY NOT NULL Auto-number starting at 1000000, +1 each row CHECK: 7 digits (1000000 to 9999999)
Name	NVARCHAR(100)	NOT NULL
Description	NVARCHAR(500)	NULL allowed (optional)
Start_date	DATE	NOT NULL
End_date	DATE	NOT NULL CHECK: End_date >= Start_date
Is_active	BIT	NOT NULL DEFAULT 1

4. Supplier

Companies that provide the products; stores their contact information.

Attribute	Type	Constraint
Supplier_id	INT IDENTITY(1000000,1)	PRIMARY KEY NOT NULL Auto-number starting at 1000000, +1 each row CHECK: 7 digits (1000000 to 9999999)
Name	NVARCHAR(100)	NOT NULL
Email	VARCHAR(100)	NOT NULL
Phone	VARCHAR(20)	NOT NULL
Street_number	NVARCHAR(100)	NOT NULL
Apt_number	VARCHAR(10)	NULL allowed (optional)
City	NVARCHAR(50)	NOT NULL
State	CHAR(2)	NOT NULL
Zip_code	VARCHAR(10)	NOT NULL

5. Register

A virtual register used to check customers out.

Attribute	Type	Constraint
Register_id	INT IDENTITY(1000000,1)	PRIMARY KEY NOT NULL Auto-number starting at 1000000, +1 each row CHECK: 7 digits (1000000 to 9999999)
Register_number	INT	NOT NULL UNIQUE
Is_active	BIT	NOT NULL DEFAULT 1

Tier 2 - need Rewards_Program / Sale_Event

6. Reward_Tier

Levels of a rewards program, each starting at a minimum number of points.

Attribute	Type	Constraint
Tier_id	INT IDENTITY(1000000,1)	PRIMARY KEY NOT NULL Auto-number starting at 1000000, +1 each row CHECK: 7 digits (1000000 to 9999999)
Program_id	INT	NOT NULL UNIQUE together with Min_points FOREIGN KEY -> Rewards_Program(Program_id)
Name	NVARCHAR(50)	NOT NULL
Min_points	INT	NOT NULL CHECK: Min_points >= 0 UNIQUE together with Program_id
Perk_description	NVARCHAR(500)	NULL allowed (optional)

7. Discount

A price reduction, either a percentage or a fixed amount, valid between two dates.

Attribute	Type	Constraint
Discount_id	INT IDENTITY(1000000,1)	PRIMARY KEY NOT NULL Auto-number starting at 1000000, +1 each row CHECK: 7 digits (1000000 to 9999999)
Name	NVARCHAR(100)	NOT NULL
Discount_type	VARCHAR(30)	NOT NULL
Code	VARCHAR(30)	NULL allowed (optional) UNIQUE among non-NULL values (filtered index)
Amount	DECIMAL(10,2)	NOT NULL CHECK: Amount >= 0
Amount_type	VARCHAR(10)	NOT NULL CHECK: Amount_type IN ('percent','fixed')
Start_date	DATE	NOT NULL
End_date	DATE	NOT NULL CHECK: End_date >= Start_date
Sale_event_id	INT	NULL allowed (optional) FOREIGN KEY -> Sale_Event(Sale_event_id)

Tier 3 - need Reward_Tier / Discount

8. Customer

Registered customers with contact data and loyalty points. A customer can be referred by another.

Attribute	Type	Constraint
Customer_id	INT IDENTITY(1000000,1)	PRIMARY KEY NOT NULL Auto-number starting at 1000000, +1 each row CHECK: 7 digits (1000000 to 9999999)
First_name	NVARCHAR(50)	NOT NULL
Last_name	NVARCHAR(50)	NOT NULL
Email	VARCHAR(100)	NOT NULL UNIQUE
Loyalty_points	INT	NOT NULL DEFAULT 0 CHECK: Loyalty_points >= 0
Street_number	NVARCHAR(100)	NOT NULL
Apt_number	VARCHAR(10)	NULL allowed (optional)
City	NVARCHAR(50)	NOT NULL
State	CHAR(2)	NOT NULL
Zip_code	VARCHAR(10)	NOT NULL
Created_date	DATE	NOT NULL DEFAULT today's date
Referred_by_id	INT	NULL allowed (optional) CHECK: Referred_by_id <> Customer_id FOREIGN KEY -> Customer(Customer_id)
Tier_id	INT	NULL allowed (optional) FOREIGN KEY -> Reward_Tier(Tier_id)

9. Category

Groups of products (consoles, discs, merch, accessories). May carry a discount.

Attribute	Type	Constraint
Category_id	INT IDENTITY(1000000,1)	PRIMARY KEY NOT NULL Auto-number starting at 1000000, +1 each row CHECK: 7 digits (1000000 to 9999999)
Name	NVARCHAR(100)	NOT NULL UNIQUE
Description	NVARCHAR(500)	NULL allowed (optional)
Discount_id	INT	NULL allowed (optional) FOREIGN KEY -> Discount(Discount_id)

Tier 4 - need Customer / Category / Discount

10. Customer_Phone

A customer can have several phone numbers. Composite primary key.

Attribute	Type	Constraint
Customer_id	INT	PRIMARY KEY (part of composite key) NOT NULL FOREIGN KEY -> Customer(Customer_id)
Phone_number	VARCHAR(20)	PRIMARY KEY (part of composite key) NOT NULL

11. Product

Items sold by the store. Every product belongs to exactly one category.

Attribute	Type	Constraint
Product_id	INT IDENTITY(1000000,1)	PRIMARY KEY NOT NULL Auto-number starting at 1000000, +1 each row CHECK: 7 digits (1000000 to 9999999)
SKU	VARCHAR(50)	NOT NULL UNIQUE
Name	NVARCHAR(150)	NOT NULL
Description	NVARCHAR(MAX)	NULL allowed (optional)
Cost_price	DECIMAL(10,2)	NOT NULL CHECK: Cost_price >= 0
Sell_price	DECIMAL(10,2)	NOT NULL CHECK: Sell_price >= 0
Stock_quantity	INT	NOT NULL DEFAULT 0 CHECK: Stock_quantity >= 0
Reorder_threshold	INT	NOT NULL CHECK: Reorder_threshold >= 0
Product_condition	VARCHAR(20)	NOT NULL CHECK: Product_condition IN ('New','Used','Refurbished')
Category_id	INT	NOT NULL FOREIGN KEY -> Category(Category_id)
Discount_id	INT	NULL allowed (optional) FOREIGN KEY -> Discount(Discount_id)

12. Tax_Rate

Tax rate that applies to a category during a period.

Attribute	Type	Constraint
Tax_rate_id	INT IDENTITY(1000000,1)	PRIMARY KEY NOT NULL Auto-number starting at 1000000, +1 each row CHECK: 7 digits (1000000 to 9999999)
Category_id	INT	NOT NULL FOREIGN KEY -> Category(Category_id)
Rate	DECIMAL(5,4)	NOT NULL CHECK: Rate BETWEEN 0 AND 1
Effective_from	DATE	NOT NULL
Effective_to	DATE	NULL allowed (optional) CHECK: Effective_to IS NULL OR Effective_to >= Effective_from

Tier 5 - need Supplier / Product / Employee

13. Supplier_Product

Which supplier can supply which product (many-to-many). Composite primary key.

Attribute	Type	Constraint
Supplier_id	INT	PRIMARY KEY (part of composite key) NOT NULL FOREIGN KEY -> Supplier(Supplier_id)
Product_id	INT	PRIMARY KEY (part of composite key) NOT NULL FOREIGN KEY -> Product(Product_id)
Supplier_sku	VARCHAR(50)	NOT NULL
Unit_cost	DECIMAL(10,2)	NOT NULL CHECK: Unit_cost >= 0
Lead_time_days	INT	NOT NULL CHECK: Lead_time_days >= 0

14. Supplier_Order

A purchase order placed by an employee with a supplier.

Attribute	Type	Constraint
Order_id	INT IDENTITY(1000000,1)	PRIMARY KEY NOT NULL Auto-number starting at 1000000, +1 each row CHECK: 7 digits (1000000 to 9999999)
Supplier_id	INT	NOT NULL FOREIGN KEY -> Supplier(Supplier_id)
Employee_id	INT	NOT NULL FOREIGN KEY -> Employee(Employee_id)
Order_date	DATE	NOT NULL
Received_date	DATE	NULL allowed (optional) CHECK: Received_date IS NULL OR Received_date >= Order_date
Status	VARCHAR(20)	NOT NULL CHECK: Status IN ('Pending','Ordered','Received','Cancelled')

Tier 6 - need Product / Supplier_Order / Employee / Customer / Register

15. Supplier_Order_Item

Products and quantities on a supplier order. Composite primary key.

Attribute	Type	Constraint
Order_id	INT	PRIMARY KEY (part of composite key) NOT NULL FOREIGN KEY -> Supplier_Order(Order_id)
Product_id	INT	PRIMARY KEY (part of composite key) NOT NULL FOREIGN KEY -> Product(Product_id)
Quantity	INT	NOT NULL CHECK: Quantity > 0
Unit_cost	DECIMAL(10,2)	NOT NULL CHECK: Unit_cost >= 0

16. Low_Stock_Alert

Raised when a product falls to its reorder threshold; resolved by a supplier order.

Attribute	Type	Constraint
Alert_id	INT IDENTITY(1000000,1)	PRIMARY KEY NOT NULL Auto-number starting at 1000000, +1 each row CHECK: 7 digits (1000000 to 9999999)
Product_id	INT	NOT NULL FOREIGN KEY -> Product(Product_id)
Triggered_at	DATETIME2	NOT NULL DEFAULT current date and time
Resolved_at	DATETIME2	NULL allowed (optional)
Order_id	INT	NULL allowed (optional) FOREIGN KEY -> Supplier_Order(Order_id)

17. Inventory_Adjustment

A manual stock correction made by an employee (quantity may be negative).

Attribute	Type	Constraint
Adjustment_id	INT IDENTITY(1000000,1)	PRIMARY KEY NOT NULL Auto-number starting at 1000000, +1 each row CHECK: 7 digits (1000000 to 9999999)
Product_id	INT	NOT NULL FOREIGN KEY -> Product(Product_id)
Employee_id	INT	NOT NULL FOREIGN KEY -> Employee(Employee_id)
Quantity_change	INT	NOT NULL CHECK: Quantity_change <> 0
Reason	NVARCHAR(200)	NOT NULL
Adjusted_at	DATETIME2	NOT NULL DEFAULT current date and time

18. Price_Change

History of price edits made by employees.

Attribute	Type	Constraint
Price_change_id	INT IDENTITY(1000000,1)	PRIMARY KEY NOT NULL Auto-number starting at 1000000, +1 each row CHECK: 7 digits (1000000 to 9999999)
Product_id	INT	NOT NULL FOREIGN KEY -> Product(Product_id)
Employee_id	INT	NOT NULL FOREIGN KEY -> Employee(Employee_id)
Old_price	DECIMAL(10,2)	NOT NULL CHECK: Old_price >= 0
New_price	DECIMAL(10,2)	NOT NULL CHECK: New_price >= 0
Changed_at	DATETIME2	NOT NULL DEFAULT current date and time

19. Sales_Transaction

One checkout. (The conceptual entity TRANSACTION; renamed because TRANSACTION is a reserved word in T-SQL.)

Attribute	Type	Constraint
Transaction_id	INT IDENTITY(1000000,1)	PRIMARY KEY NOT NULL Auto-number starting at 1000000, +1 each row CHECK: 7 digits (1000000 to 9999999)
Customer_id	INT	NULL allowed (optional) FOREIGN KEY -> Customer(Customer_id)
Employee_id	INT	NOT NULL FOREIGN KEY -> Employee(Employee_id)
Register_id	INT	NOT NULL FOREIGN KEY -> Register(Register_id)
Transaction_date	DATETIME2	NOT NULL DEFAULT current date and time
Subtotal	DECIMAL(10,2)	NOT NULL DEFAULT 0 CHECK: Subtotal >= 0
Tax_amount	DECIMAL(10,2)	NOT NULL DEFAULT 0 CHECK: Tax_amount >= 0
Total	DECIMAL(10,2)	NOT NULL DEFAULT 0 CHECK: Total >= 0
Status	VARCHAR(10)	NOT NULL DEFAULT 'open' CHECK: Status IN ('open','completed','voided')

Tier 7 - need Sales_Transaction

20. Transaction_Item

One product line of a transaction. Composite primary key.

Attribute	Type	Constraint
Transaction_id	INT	PRIMARY KEY (part of composite key) NOT NULL FOREIGN KEY -> Sales_Transaction(Transaction_id)
Product_id	INT	PRIMARY KEY (part of composite key) NOT NULL FOREIGN KEY -> Product(Product_id)
Quantity	INT	NOT NULL CHECK: Quantity > 0
Unit_price	DECIMAL(10,2)	NOT NULL CHECK: Unit_price >= 0
Discount_id	INT	NULL allowed (optional) FOREIGN KEY -> Discount(Discount_id)
Discount_amount	DECIMAL(10,2)	NOT NULL DEFAULT 0 CHECK: Discount_amount >= 0 AND Discount_amount <= Quantity * Unit_price
Line_total	Computed (DECIMAL)	Calculated by SQL Server (stored). Do not INSERT it. Formula: Quantity * Unit_price - Discount_amount

21. Payment

How a transaction was paid. Only the last 4 card digits are ever stored.

Attribute	Type	Constraint
Payment_id	INT IDENTITY(1000000,1)	PRIMARY KEY NOT NULL Auto-number starting at 1000000, +1 each row CHECK: 7 digits (1000000 to 9999999)
Transaction_id	INT	NOT NULL FOREIGN KEY -> Sales_Transaction(Transaction_id)
Method	VARCHAR(10)	NOT NULL CHECK: Method IN ('credit','debit','cash')
Amount	DECIMAL(10,2)	NOT NULL CHECK: Amount > 0
Card_last4	CHAR(4)	NULL allowed (optional) CHECK: Card_last4 IS NULL OR Card_last4 NOT LIKE '%[^0-9]%'
Change_given	DECIMAL(10,2)	NOT NULL DEFAULT 0 CHECK: Change_given >= 0
Paid_at	DATETIME2	NOT NULL DEFAULT current date and time

22. Review

A customer rating (1 to 5 stars) of a product, optionally tied to a transaction.

Attribute	Type	Constraint
Review_id	INT IDENTITY(1000000,1)	PRIMARY KEY NOT NULL Auto-number starting at 1000000, +1 each row CHECK: 7 digits (1000000 to 9999999)
Rating	TINYINT	NOT NULL CHECK: Rating BETWEEN 1 AND 5
Comment	NVARCHAR(1000)	NULL allowed (optional)
Review_date	DATETIME2	NOT NULL DEFAULT current date and time
Product_id	INT	NOT NULL FOREIGN KEY -> Product(Product_id)
Customer_id	INT	NOT NULL FOREIGN KEY -> Customer(Customer_id)
Transaction_id	INT	NULL allowed (optional) FOREIGN KEY -> Sales_Transaction(Transaction_id)

23. Loyalty_Transaction

Points earned or redeemed by a customer under a rewards program.

Attribute	Type	Constraint
Loyalty_txn_id	INT IDENTITY(1000000,1)	PRIMARY KEY NOT NULL Auto-number starting at 1000000, +1 each row CHECK: 7 digits (1000000 to 9999999)
Customer_id	INT	NOT NULL FOREIGN KEY -> Customer(Customer_id)
Transaction_id	INT	NULL allowed (optional) FOREIGN KEY -> Sales_Transaction(Transaction_id)
Program_id	INT	NOT NULL FOREIGN KEY -> Rewards_Program(Program_id)
Points_change	INT	NOT NULL
Reason	NVARCHAR(100)	NOT NULL
Created_at	DATETIME2	NOT NULL DEFAULT current date and time

Part 3 - SQL for each data type and each constraint

Everything below is the exact syntax used in schema.sql. Each constraint is shown two ways: written inside CREATE TABLE (how the schema does it) and added to an existing table with ALTER TABLE (useful if the design changes).

A. Data types

Type	Stores	Column definition (from the schema)	Example value
INT	whole numbers	Quantity INT NOT NULL	3
INT IDENTITY	automatic 7-digit ID	Product_id INT IDENTITY(1000000,1) NOT NULL	leave out of INSERT
TINYINT	whole numbers 0-255	Rating TINYINT NOT NULL	5
DECIMAL(10,2)	exact money, 2 decimals	Sell_price DECIMAL(10,2) NOT NULL	59.99
DECIMAL(5,4)	tax rate as a fraction	Rate DECIMAL(5,4) NOT NULL	0.0825 (8.25%)
VARCHAR(n)	plain text, up to n characters	Email VARCHAR(100) NOT NULL	'ana@store.com'
NVARCHAR(n)	Unicode text (accents, other alphabets)	Name NVARCHAR(150) NOT NULL	N'Pokémon Legends'
NVARCHAR(MAX)	text of any length	Description NVARCHAR(MAX) NULL	N'Long text...'
CHAR(n)	text of exactly n characters	State CHAR(2) NOT NULL	'TX'
DATE	a date, no time	Hire_date DATE NOT NULL	'2026-01-15'
DATETIME2	date and time	Paid_at DATETIME2 NOT NULL DEFAULT SYSDATETIME()	'2026-09-29 14:30:00'
BIT	yes/no (0 or 1)	Is_active BIT NOT NULL DEFAULT 1	1
Computed column	value calculated by SQL Server	Line_total AS (Quantity * Unit_price - Discount_amount) PERSISTED	never inserted

- **Money is always DECIMAL, never FLOAT**: FLOAT gives rounding errors such as 0.1 + 0.2.
- **Zip codes and phone numbers are text** (VARCHAR), so leading zeros and + signs are kept.
- **Text with accents needs the N prefix** on insert: N'Pokémon'. Without it the accent can be lost.
- **Never insert into an IDENTITY or computed column**; list only the other columns.

Putting the types together in an INSERT (the ID columns are left out, SQL Server fills them in):

```
INSERT INTO Employee
    (First_name, Last_name, Email, Phone_number, Role, Hire_date, Password_hash)
VALUES
    (N'Ana', N'López', 'ana@store.com', '713-555-0101', 'Manager', '2026-01-15', '$2b$10$exampleHash');

INSERT INTO Register (Register_number, Is_active) VALUES (1, 1);

-- read back the automatic ID that was generated
SELECT Employee_id, First_name FROM Employee;       -- Employee_id = 1000000
```

Delete any test rows afterwards. To make an empty table number from 1000000 again: `DBCC CHECKIDENT ('Employee', RESEED, 999999);`

B. How each constraint is implemented

1. NOT NULL / NULL (mandatory or optional attribute)

```
-- inside CREATE TABLE
Phone_number  VARCHAR(20)  NULL,            -- optional
Email         VARCHAR(100) NOT NULL,        -- mandatory

-- on an existing table (fails if the column already contains NULLs)
ALTER TABLE Employee ALTER COLUMN Phone_number VARCHAR(20) NOT NULL;
```

2. DEFAULT (value used when none is given)

```
-- inside CREATE TABLE
Stock_quantity  INT       NOT NULL DEFAULT 0,
Status          VARCHAR(10) NOT NULL DEFAULT 'open',
Paid_at         DATETIME2 NOT NULL DEFAULT SYSDATETIME(),

-- on an existing table
ALTER TABLE Product ADD CONSTRAINT DF_Product_Stock DEFAULT 0 FOR Stock_quantity;
```

3. IDENTITY + 7-digit rule (automatic, unique IDs)

```
Employee_id  INT IDENTITY(1000000,1) NOT NULL,      -- first ID 1000000, then +1
CONSTRAINT CK_Employee_Id_7digits CHECK (Employee_id BETWEEN 1000000 AND 9999999)
```

IDENTITY numbers rows but is not what forbids duplicates; the PRIMARY KEY does. Gaps after failed inserts or deletes are normal. An existing column cannot be turned into IDENTITY: drop the table and recreate it.

4. PRIMARY KEY (single and composite)

```
-- single column
CONSTRAINT PK_Employee PRIMARY KEY (Employee_id)

-- two columns together identify the row (Customer_Phone, Transaction_Item, ...)
CONSTRAINT PK_Transaction_Item PRIMARY KEY (Transaction_id, Product_id)
```

5. FOREIGN KEY (relationships)

```
-- inside CREATE TABLE: every product has exactly one category (column is NOT NULL)
Category_id  INT NOT NULL,
CONSTRAINT FK_Product_Category FOREIGN KEY (Category_id) REFERENCES Category(Category_id)

-- optional relationship: make the column NULL (a walk-in sale has no customer)
Customer_id  INT NULL,
CONSTRAINT FK_Sales_Transaction_Customer FOREIGN KEY (Customer_id) REFERENCES Customer(Customer_id)

-- self-reference: supervisor is another employee, a customer refers another customer
CONSTRAINT FK_Employee_Supervisor FOREIGN KEY (Supervisor_id) REFERENCES Employee(Employee_id)

-- on an existing table
ALTER TABLE Product ADD CONSTRAINT FK_Product_Category
    FOREIGN KEY (Category_id) REFERENCES Category(Category_id);
```

The parent table must exist first (tier order). A row cannot be deleted while another row references it, because no cascade is used.

6. UNIQUE

```
-- one column
CONSTRAINT UQ_Employee_Email UNIQUE (Email)

-- combination of columns: no two tiers of a program start at the same points
CONSTRAINT UQ_Reward_Tier_Program_Min UNIQUE (Program_id, Min_points)

-- optional column that must still be unique when present (a plain UNIQUE allows only one NULL)
CREATE UNIQUE INDEX UX_Discount_Code ON Discount(Code) WHERE Code IS NOT NULL;

-- on an existing table
ALTER TABLE Customer ADD CONSTRAINT UQ_Customer_Email UNIQUE (Email);
```

7. CHECK (rules on values)

```
CONSTRAINT CK_Product_Price     CHECK (Sell_price >= 0)                    -- not negative
CONSTRAINT CK_Review_Rating     CHECK (Rating BETWEEN 1 AND 5)             -- range
CONSTRAINT CK_Employee_Role     CHECK (Role IN ('Manager','Cashier'))      -- allowed list
CONSTRAINT CK_Sale_Event_Dates  CHECK (End_date >= Start_date)             -- two columns of one row
CONSTRAINT CK_Program_Dates     CHECK (Effective_to IS NULL OR Effective_to >= Effective_from)
                                                                           -- optional end date
CONSTRAINT CK_Employee_NotOwnSupervisor CHECK (Supervisor_id <> Employee_id)  -- not themselves
CONSTRAINT CK_Payment_Card      CHECK (Card_last4 IS NULL OR Card_last4 NOT LIKE '%[^0-9]%')
                                                                           -- digits only

-- on an existing table (existing rows are checked; it fails if any row breaks the rule)
ALTER TABLE Product ADD CONSTRAINT CK_Product_Price CHECK (Sell_price >= 0);
```

A CHECK on an optional column must allow NULL explicitly (`IS NULL OR ...`), because a comparison with NULL is never true. A CHECK can only look at the row being written.

8. Rules that involve other rows: triggers

When a rule needs other rows or tables, use a trigger, which SQL Server runs automatically. The pseudo-tables `inserted` and `deleted` hold the new and old rows. Example: an employee's supervisor must be a Manager.

```
CREATE OR ALTER TRIGGER trg_Employee_SupervisorIsManager
ON Employee
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF NOT (UPDATE(Supervisor_id) OR UPDATE(Role)) RETURN;

    IF EXISTS (
        SELECT 1
        FROM Employee e
        JOIN Employee s ON s.Employee_id = e.Supervisor_id
        WHERE s.Role <> 'Manager'
          AND (   e.Employee_id IN (SELECT Employee_id FROM inserted)
               OR s.Employee_id IN (SELECT Employee_id FROM inserted))
    )
    BEGIN
        ROLLBACK TRANSACTION;                       -- undo the offending change
        THROW 50001, 'An employee''s supervisor must hold the Manager role.', 1;
    END
END;
GO   -- CREATE TRIGGER must be the first statement of its batch
```

Rule from the conceptual schema	Table	How it is enforced
A supervisor must hold the Manager role	Employee	trigger (above)
Items can be added or removed only while the transaction is open	Transaction_Item	trigger: reject if the transaction Status is not open

Rule from the conceptual schema	Table	How it is enforced
A product cannot be sold if stock is lower than the quantity requested	Transaction_Item	trigger: compare quantity with Product.Stock_quantity
Stock decreases only when a transaction becomes completed; voided leaves stock unchanged	Sales_Transaction	trigger on Status change
Marking a supplier order received increases each product's stock	Supplier_Order	trigger on Status change to Received
Only managers edit inventory, prices and discounts	-	application layer (Node.js checks the logged-in role)
A customer's tier must match their points	-	application layer (recalculated when points change)

If triggers change stock, the application must only change a transaction's Status and never update Stock_quantity itself, or stock would change twice.

C. Testing a constraint and reading its errors

Try a value that should be rejected; SQL Server names the constraint that stopped it.

```
-- fails with error 547, constraint CK_Employee_Role
INSERT INTO Employee (First_name, Last_name, Email, Role, Hire_date, Password_hash)
VALUES (N'Test', N'User', 'test@example.com', 'Boss', '2026-01-01', 'x');

-- fails with error 547, constraint CK_Sale_Event_Dates
INSERT INTO Sale_Event (Name, Start_date, End_date)
VALUES (N'Bad dates', '2026-02-01', '2026-01-01');
```

Error	Meaning	Typical cause
515	Cannot insert NULL	a NOT NULL column was left out
547	CHECK or FOREIGN KEY conflict	value breaks a rule, or the referenced parent row does not exist
2627	PRIMARY KEY or UNIQUE constraint violated	duplicate key or duplicate email/SKU
2601	Duplicate value in a unique index	duplicate discount code

See the constraints of one table, or all CHECK rules:

```
EXEC sp_help 'Product';                     -- columns, keys, constraints of one table

SELECT OBJECT_NAME(parent_object_id) AS table_name, name, definition
FROM sys.check_constraints ORDER BY table_name, name;
```